

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1710

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

*I **BHUVANESHWARAN J(192511402)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: *Bhuvaneshwaran J*

Student 4: : Bhuvaneshwaran J | Register Number: 192511402

Reverse Engineering a Social Media Content Moderation System

A social media platform automatically flags and removes harmful content within seconds of posting. Reverse

engineer the system design by identifying how content data (text, images) is processed using RDD transformations,

and explain how intelligent agents contribute to real-time moderation decisions and escalation to human reviewers.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

SIMATS ENGINEERING

ASSESSMENT TOOL 3 — Answer Script (Annexure A: Reverse Engineering)

Course Name:	Artificial Intelligence	Course Code:	CSA1710
Student Name:	Bhuvaneshwaran J	Register No.:	192511402
Course Outcome:	CO5 (BTL 5)	Weightage:	20%

Topic: Reverse Engineering a Social Media Content Moderation System

Problem statement given: A social media platform automatically flags and removes harmful content within seconds of posting. Reverse engineer the system design by identifying how content data (text, images) is processed using RDD transformations, and explain how intelligent agents contribute to real-time moderation decisions and escalation to human reviewers.

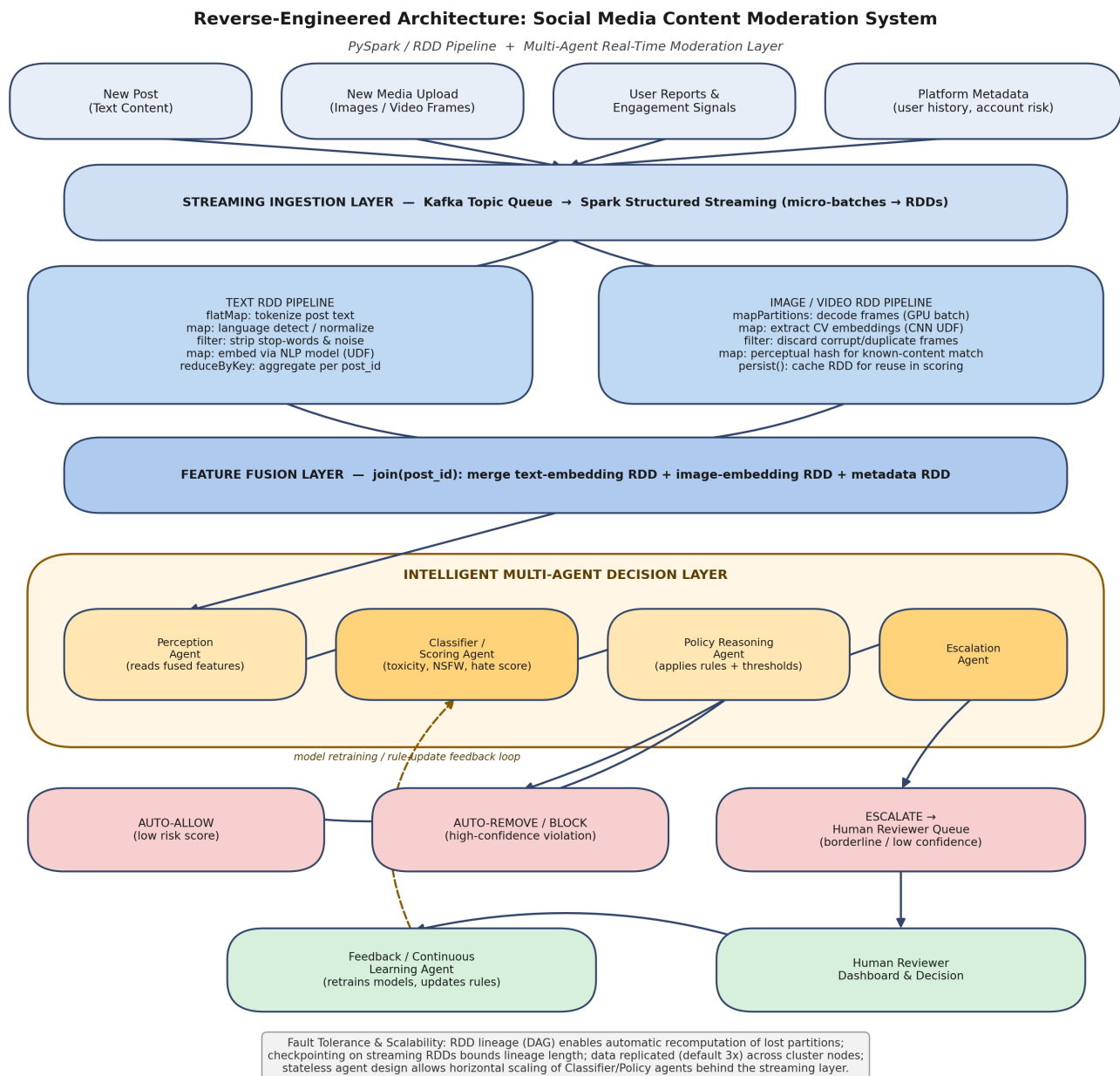
1. Understanding the Industry Problem

Content moderation platforms must inspect an extremely high volume of heterogeneous, continuously arriving data — text posts, comments, images, video frames, and user-behaviour signals — and decide, within seconds, whether each item is safe, violates policy, or is ambiguous enough to need a person's judgement. From only the system's observable behaviour (fast automated takedowns, occasional delayed removals after a report, and visible appeal/review outcomes), several hard constraints can be inferred:

- **Extreme throughput:** millions of posts/images per hour must be scored in near real time.
- **Low latency:** harmful content is often removed within seconds, implying streaming (not batch-only) processing.
- **Multi-modality:** text and image/video pipelines run in parallel and their outputs must be fused per post.
- **Fault tolerance:** node failures during peak load cannot cause missed detections or duplicate actions.
- **Human-in-the-loop:** borderline cases are escalated rather than auto-decided, balancing automation with accountability.

2. Reverse-Engineered System Architecture

Working backward from these constraints, the moderation pipeline can be reconstructed as a streaming PySpark system feeding a layer of cooperating intelligent agents, as shown below.



3. Data Pipeline Using PySpark and RDD Operations

New posts arrive on a message queue (e.g., Kafka) and are consumed by Spark Structured Streaming, which discretises the stream into micro-batch RDDs. Two parallel RDD pipelines transform raw content into feature vectors that the agent layer can reason about.

3.1 Text Pipeline (inferred RDD transformations)

RDD Operation	Inferred Purpose
flatMap	Tokenize each post into words/sub-word units.
map	Detect language, normalize case/unicode, strip emoji noise.
filter	Discard stop-words and empty tokens before scoring.
map (UDF)	Convert cleaned tokens into a dense embedding via an NLP model.
reduceByKey	Aggregate token-level embeddings back to one vector per post_id.

3.2 Image / Video Pipeline (inferred RDD transformations)

RDD Operation	Inferred Purpose
mapPartitions	Decode image/video frames in batches per partition for GPU efficiency.
map (UDF)	Extract CNN-based visual embeddings (objects, nudity/violence indicators).
filter	Drop corrupted, duplicate, or already-known-safe frames using a hash lookup.
map	Compute a perceptual hash to match against known-violating content databases.
persist() / cache()	Cache the embedding RDD so it can be reused by multiple scoring agents without recomputation.

3.3 Fusion and Fault Tolerance

The two RDDs are combined with a **join(post_id)** transformation together with platform metadata (account risk score, report count) to build one feature record per post. Because RDDs are immutable and track lineage (the DAG of transformations that produced them), Spark can automatically recompute only the lost partitions if a worker node fails, rather than losing the whole batch. Periodic **checkpointing** on the streaming RDDs bounds this lineage so recovery stays fast even after many micro-batches, which explains why the platform keeps functioning smoothly during infrastructure failures without visibly dropping moderation coverage.

4. Role of Intelligent Agents in Real-Time Moderation

The feature RDD is handed to a layer of cooperating agents. Each agent has a narrow perception-decision-action loop, and together they form a multi-agent system that mirrors how the platform appears to behave from the outside (instant blocks for obvious violations, short delays for ambiguous ones, and visible appeal outcomes days later).

Agent	Perceives	Decides / Acts
Perception Agent	Fused feature RDD (text + image + metadata)	Structures raw features into a standard case object for downstream agents.
Classifier / Scoring Agent	Case object	Runs ML models to output toxicity, hate-speech, NSFW and violence scores with confidence values.
Policy Reasoning Agent	Scores + platform policy rules	Applies thresholds and business rules (e.g., context, region-specific law) to choose allow / remove / escalate.
Escalation Agent	Borderline / low-confidence cases	Routes the case to the human reviewer queue with priority ranking and supporting evidence.
Feedback / Learning Agent	Human reviewer verdicts	Feeds corrected labels back to retrain classifiers and update policy thresholds continuously.

4.1 Real-Time Decision Workflow

- 1 Post is ingested and converted into text/image RDDs within the same micro-batch window (sub-second to low-second latency).
- 2 Perception Agent assembles a unified case; Classifier Agent scores it against multiple risk dimensions in parallel.
- 3 Policy Agent applies threshold logic: score above a high-confidence violation threshold → **auto-remove**; below a safe threshold → **auto-allow**; in between → **escalate**.
- 4 Escalated cases enter a priority queue (ranked by potential harm and virality) surfaced on the Human Reviewer Dashboard.
- 5 Reviewer decisions are captured by the Feedback Agent and streamed back to retrain the Classifier Agent, closing the loop.

5. Scalability, Fault Tolerance and Evaluation

The design is horizontally scalable because RDD partitions can be distributed across additional worker nodes as post volume spikes (e.g., during breaking news), and the agents are stateless consumers of feature records, so more agent instances can be added behind the streaming layer without redesigning the pipeline. To validate such a reverse-engineered design, the system's real-world performance would be evaluated using:

- **Precision / Recall** of automated removals against reviewer-confirmed ground truth.
- **End-to-end latency** from post creation to moderation decision.
- **Escalation rate and reviewer agreement** — how often human review overturns the automated verdict.
- **Recovery time** after simulated node failure, to confirm the fault-tolerance claims above.

6. Conclusion

By observing the platform's near-instant takedowns, occasional escalations, and resilience under load, the underlying design can be reconstructed as a Spark Structured Streaming pipeline that converts multi-modal content into RDD-based feature vectors, which are then consumed by a coordinated set of perception, classification, policy, escalation, and learning agents. This combination explains both the system's speed (automated, parallelized RDD processing) and its accountability (human review for ambiguous cases with a continuous feedback loop back into the models).

Declaration: This submission represents original analysis prepared for CO5 – Assessment Tool 3 (Reverse Engineering, Annexure A) and adheres to the academic integrity guidelines stated on the cover page.